

03_mock: Reliability-Focused Incident Triage Agent

Timebox: **55 minutes**

Language: **Python (Colab)**

Scenario

You are given an incident triage loop with flaky tools. Add reliability controls expected in production-like agent systems.

Progressive requirements

1. Tool execution with validation
2. Cache for repeated tool inputs
3. One retry for transient runtime failures
4. Claude-style sequencing (`assistant tool_use -> user tool_result`)
5. Structured final output (`summary` , `action` , `confidence`)

What to implement

- `execute_tool_call`
- `parse_final_output`
- `run_agent`

Time guidance

- 10 min: map retry/cache behavior
- 35 min: implement loop + parsing
- 10 min: run tests + verify stats

```
In [ ]: import inspect
import json
from copy import deepcopy
from typing import Any, Callable

LOOKUP_ATTEMPTS: dict[str, int] = {}

def reset_state() -> None:
    LOOKUP_ATTEMPTS.clear()
```

```

def fetch_ticket(ticket_id: str) -> dict[str, Any]:
    return {
        "ticket_id": ticket_id,
        "service": "payments" if ticket_id == "inc-1" else "billing",
        "severity": "high",
    }

def lookup_runbook(service: str) -> dict[str, Any]:
    count = LOOKUP_ATTEMPTS.get(service, 0)
    LOOKUP_ATTEMPTS[service] = count + 1
    if service == "payments" and count == 0:
        raise RuntimeError("transient backend timeout")
    return {"service": service, "playbook": f"restart_{service}_workers"}

TOOL_REGISTRY: dict[str, Callable[..., Any]] = {
    "fetch_ticket": fetch_ticket,
    "lookup_runbook": lookup_runbook,
}

class TriageModel:
    def __init__(self, scenario: str) -> None:
        self.scenario = scenario
        self.step = 0

    def __call__(self, messages: list[dict[str, Any]]) -> dict[str, Any]:
        self.step += 1

        if self.scenario == "cache":
            if self.step == 1:
                return {
                    "stop_reason": "tool_use",
                    "content": [
                        {"type": "tool_use", "id": "a1", "name": "lookup_run"},
                    ],
                }
            if self.step == 2:
                return {
                    "stop_reason": "tool_use",
                    "content": [
                        {"type": "tool_use", "id": "a2", "name": "lookup_run"},
                    ],
                }
            return {
                "stop_reason": "end_turn",
                "content": [
                    {
                        "type": "text",
                        "text": json.dumps(
                            {"summary": "billing issue mitigated", "action":

```

```

    }

    if self.scenario == "retry":
        if self.step == 1:
            return {
                "stop_reason": "tool_use",
                "content": [
                    {"type": "tool_use", "id": "b1", "name": "lookup_run"},
                ],
            }
            return {
                "stop_reason": "end_turn",
                "content": [
                    {
                        "type": "text",
                        "text": json.dumps(
                            {"summary": "payments issue mitigated", "action": ""}
                        ),
                    },
                ],
            }

        if self.scenario == "loop":
            return {
                "stop_reason": "tool_use",
                "content": [{"type": "tool_use", "id": "loop", "name": "fetch"}]
            }

    return {"stop_reason": "end_turn", "content": [{"type": "text", "text": ""}]}

```

```

In [ ]: def execute_tool_call(
    tool_call: dict[str, Any],
    tool_registry: dict[str, Callable[..., Any]],
    cache: dict[str, dict[str, Any]],
) -> dict[str, Any]:
    """Execute tool call with cache + one retry for transient RuntimeError."""
    # TODO:
    # - Validate required fields and args.
    # - Cache key = name + stable JSON input.
    # - Use cache for repeated calls (set from_cache=True).
    # - Retry once on transient RuntimeError.
    # - Return tool_result block: type/tool_use_id/is_error/content/from_cache
    raise NotImplementedError

def parse_final_output(output_text: str) -> dict[str, Any]:
    """Parse JSON output and validate summary/action/confidence fields."""
    # TODO:
    # - Parse JSON.
    # - Ensure keys summary/action/confidence exist.
    # - Ensure confidence is numeric.
    raise NotImplementedError

def run_agent(
    user_prompt: str,

```

```

    model: Callable[[list[dict[str, Any]], dict[str, Any]],
    tool_registry: dict[str, Callable[..., Any]],
    max_steps: int = 6,
) -> dict[str, Any]:
    """Run reliability-focused Claude-style loop and return final + stats."""
    # TODO:
    # - Track stats: tool_calls, cache_hits.
    # - Append assistant content each turn.
    # - On tool_use, execute all tool_use blocks and append one user tool_re
    # - On end_turn, return parsed final output + stats + messages.
    # - Raise RuntimeError("max_steps_exceeded") on exhaustion.
    raise NotImplementedError

```

Run Tests

Run this final test cell after implementing all TODO sections.

```

In [ ]: def _tool_result_blocks(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    blocks: list[dict[str, Any]] = []
    for message in messages:
        if message.get("role") != "user":
            continue
        for block in message.get("content", []):
            if block.get("type") == "tool_result":
                blocks.append(block)
    return blocks

def run_exam03_tests() -> None:
    reset_state()

    # 1) Cache behavior
    model = TriageModel("cache")
    result = run_agent("triage billing", model, TOOL_REGISTRY)
    assert result["stats"]["cache_hits"] == 1
    assert LOOKUP_ATTEMPTS["billing"] == 1
    assert len(_tool_result_blocks(result["messages"])) == 2

    # 2) Retry behavior for transient errors
    reset_state()
    model = TriageModel("retry")
    result = run_agent("triage payments", model, TOOL_REGISTRY)
    assert LOOKUP_ATTEMPTS["payments"] == 2
    assert result["final"]["action"] == "restart_payments_workers"

    # 3) Structured final output required
    assert {"summary", "action", "confidence"}.issubset(result["final"].keys())

    # 4) Claude sequencing: assistant tool_use immediately followed by user
    assistant_idx = next(i for i, m in enumerate(result["messages"]) if m["role"] == "assistant")
    assert result["messages"][assistant_idx + 1]["role"] == "user"
    assert all(
        block.get("type") == "tool_result" for block in result["messages"][assistant_idx:]
    )

```

```
# 5) Max step protection
model = TriageModel("loop")
try:
    run_agent("loop", model, TOOL_REGISTRY, max_steps=3)
    raise AssertionError("Expected max_steps_exceeded")
except RuntimeError as exc:
    assert "max_steps_exceeded" in str(exc)

print("03_mock tests passed")

run_exam03_tests()
```